

Object Detection Models:

RTMDet vs YOLOv10n

Topic: Object Detection Model Comparison

Models: RTMDet (OpenMMLab) & YOLOv10n

Purpose: Academic / Project Reference

Contents

1	Introduction	3
1.1	What Is Object Detection?	3
1.2	Why Is Object Detection Important?	3
1.3	Scope of This Report	4
2	RTMDet — Real-Time Multi-Scale Detector	4
2.1	What Is RTMDet?	4
2.2	Architecture Overview	4
2.2.1	Backbone — Feature Extraction	4
2.2.2	Neck — Multi-Scale Feature Fusion	5
2.2.3	Head — Prediction	5
2.3	Anchor-Free Detection	5
2.4	Training Strategy — Dynamic Label Assignment	6
2.5	Strengths and Weaknesses of RTMDet	6
3	YOLOv10n — You Only Look Once, Version 10 (Nano)	6
3.1	What Is YOLOv10n?	6
3.2	Architecture of YOLOv10n	7
3.2.1	Backbone	7
3.2.2	Neck	7
3.2.3	Head	7
3.3	What Is NMS (Non-Maximum Suppression)?	7
3.4	How YOLOv10 Removes NMS — Dual Assignment	8
3.5	Strengths and Weaknesses of YOLOv10n	9
4	Comparison Between RTMDet and YOLOv10n	9
4.1	Overview	9
4.2	Comparison Table	10
4.3	Accuracy	11
4.4	Speed	11
4.5	Complexity and Ease of Use	11
5	Application to Seatbelt Detection	11
5.1	Why Seatbelt Detection?	11

5.2	A Simple Seatbelt Detection Pipeline (implemented)	12
5.3	Which Model Is More Suitable?	12
6	Performance on Edge Hardware	13
6.1	What Are Edge Devices?	13
6.2	Inference Speed Comparison	14
6.3	Memory Usage	14
6.4	Power Efficiency	14
6.4.1	Optimisation Tips for Edge Deployment	15
7	Conclusion	15
7.1	Summary of Key Insights	15
	References	17

1 Introduction

1.1 What Is Object Detection?

Imagine you are looking at a photograph of a busy road. Your brain immediately recognises the cars, pedestrians, traffic lights, and signs in the image. **Object detection** is the task of teaching a computer to do exactly the same thing.

More precisely, an object detection system:

1. **Locates** every object of interest in an image by drawing a tight rectangular box around it. This box is called a *bounding box*.
2. **Classifies** what is inside that box, for example: “car”, “person”, or “seatbelt”.
3. **Scores** its own confidence, saying something like “I am 92% sure this is a car”.

Analogy

Think of a bounding box like the coloured rectangle you see on your phone’s camera when it detects a face before taking a photo. Object detection generalises this idea to *any* kind of object.

1.2 Why Is Object Detection Important?

Object detection is one of the most impactful areas of computer vision. Below are just a few real-world applications:

- **Autonomous Driving:** Self-driving cars must constantly detect other vehicles, cyclists, and pedestrians to navigate safely.
- **Medical Imaging:** Detecting tumours or anomalies in X-rays and MRI scans.
- **Surveillance & Safety:** Identifying whether workers on a construction site are wearing helmets.
- **Seatbelt Detection:** Monitoring passengers to ensure they are wearing seatbelts — a key focus of this report.
- **Retail:** Counting products on shelves or detecting shoplifting.

1.3 Scope of This Report

This report compares two modern, widely used object detection models:

- **RTMDet** — a real-time detector from the OpenMMLab ecosystem.
- **YOLOv10n** — the nano (smallest) variant of YOLOv10, designed for speed and efficiency.

The comparison focuses on architecture, accuracy, speed, and their suitability for practical tasks such as seatbelt detection.

2 RTMDet — Real-Time Multi-Scale Detector

2.1 What Is RTMDet?

RTMDet stands for *Real-Time Multi-scale Detector*. It was developed by researchers at OpenMMLab (an open-source computer vision platform) and published in 2022. The goal was to build a detector that is both *accurate* and *fast enough for real-time use*.

“Real-time” means the model can analyse video frames as they arrive — typically at 30 or more frames per second — without falling behind.

2.2 Architecture Overview

A deep-learning object detector is usually divided into three main parts, often called the **backbone**, the **neck**, and the **head**. Think of them like the stages of an assembly line.

Input Image → **Backbone** → **Neck** → **Head** → **Detections**

2.2.1 Backbone — Feature Extraction

The backbone is the first part to process the image. Its job is to extract *features* — patterns like edges, textures, and shapes — from the raw pixel values.

RTMDet uses a backbone called **CSPNeXt** (Cross-Stage Partial Network, Next version). It works by repeatedly applying convolution operations to the image, progressively shrinking its spatial size while increasing the number of feature channels. The result is a rich, compact representation of the image content.

Analogy — Backbone

Imagine reading a book. The backbone is like your brain picking out the key sentences and ideas from every paragraph, ignoring irrelevant words.

2.2.2 Neck — Multi-Scale Feature Fusion

Objects in an image can be very small (a distant car) or very large (a truck close to the camera). Processing the image at only one resolution would make it hard to detect both. The **neck** solves this by combining feature maps from *different scales*.

RTMDet uses an **Enhanced FPN** (Feature Pyramid Network). It merges low-resolution features (good for large objects) with high-resolution features (good for small objects), producing a set of multi-scale feature maps for the head to use.

Analogy — Neck

Think of a telescope with multiple zoom levels. The neck is like combining what you see at 2×, 5×, and 10× zoom into one merged view, so you can spot both distant stars and nearby birds simultaneously.

2.2.3 Head — Prediction

The head takes the multi-scale feature maps and produces the final output: bounding box coordinates and class labels. RTMDet uses a **shared head**, meaning the same set of prediction layers is reused across all scale levels, which reduces the total number of parameters.

2.3 Anchor-Free Detection

Traditional detectors (such as YOLOv3) use **anchors** — pre-defined rectangular boxes of fixed sizes and ratios. The model learns to adjust these anchors to fit the actual objects. While effective, anchors introduce many hyper-parameters that are tricky to tune.

RTMDet is **anchor-free**. Instead of pre-defining box shapes, it directly predicts the distance from each grid point to the four sides of the bounding box. This simplifies the design and makes the model easier to apply to new datasets.

Key Idea

No anchors = fewer settings to configure = simpler training.

2.4 Training Strategy — Dynamic Label Assignment

During training, the model needs to know which ground-truth objects each predicted box is *responsible* for. RTMDet uses a strategy called **SimOTA** (Simplified Optimal Transport Assignment).

In plain words, instead of deciding this assignment with fixed rules, SimOTA *dynamically* figures out the best matching between predicted boxes and real objects at each training step. This leads to better gradient signals and, ultimately, higher accuracy.

2.5 Strengths and Weaknesses of RTMDet

	Strengths	Weaknesses
Accuracy	High accuracy, especially on small objects	Slightly slower than the smallest YOLO variants
Flexibility	Works well with instance segmentation too	Requires OpenMMLab (MMDetection) setup
Training	Dynamic label assignment improves convergence	Training pipeline can be complex for beginners
Anchors	Anchor-free — easier to configure	—

3 YOLOv10n — You Only Look Once, Version 10 (Nano)

3.1 What Is YOLOv10n?

YOLO stands for *You Only Look Once*. The name captures the core idea: instead of looking at the same image multiple times (like older two-stage detectors), YOLO processes the entire image in a *single forward pass* through the neural network.

YOLOv10 is the tenth major iteration of this family, published in 2024. The “n” suffix stands for **nano**, the smallest and fastest variant in the YOLOv10 lineup. It is designed to run on devices with limited computing power, such as smartphones or embedded boards.

3.2 Architecture of YOLOv10n

Like RTMDet, YOLOv10n follows the backbone–neck–head structure:

3.2.1 Backbone

YOLOv10n uses a lightweight backbone inspired by **CSPNet** and **RepConv** (Reparameterisable Convolution). RepConv is a clever trick: during training, multiple convolution branches are used to capture richer features; at inference time, these branches are mathematically *merged* into a single convolution, making the model faster.

3.2.2 Neck

The neck in YOLOv10 uses a **PAN-FPN** (Path Aggregation Network combined with Feature Pyramid Network). This bidirectional feature flow — both top-down and bottom-up — helps the model detect objects at multiple scales efficiently.

3.2.3 Head

YOLOv10 introduces a **dual head** design (explained in Section 3.4), which allows it to remove the post-processing step called NMS during inference.

3.3 What Is NMS (Non-Maximum Suppression)?

When an object detector processes an image, it often produces *many* overlapping bounding boxes for the same object. For example, it might draw 10 slightly different boxes around the same car.

Non-Maximum Suppression (NMS) is a post-processing step that filters these duplicates. It keeps only the box with the *highest confidence score* and discards all other boxes that overlap with it by more than a set threshold (called IoU threshold).

Analogy — NMS

Imagine ten people all raise their hands to answer the same question. NMS is like the teacher picking only the person whose hand is raised the highest and ignoring everyone else.

While NMS works well, it introduces *extra computation time* and has its own hyper-parameters (the IoU and score thresholds). This becomes a bottleneck, especially on hardware with limited resources.

3.4 How YOLOv10 Removes NMS — Dual Assignment

YOLOv10 introduces a creative solution called **dual label assignment**, which trains the model to avoid producing duplicate predictions in the first place.

During training, two assignment strategies run in parallel:

- **One-to-many assignment:** Multiple predicted boxes are matched to the same ground-truth object. This gives the model many training examples per object, helping it learn rich features. However, at inference time it produces duplicates.
- **One-to-one assignment:** Each ground-truth object is matched to *exactly one* predicted box. This is the head used at inference, and since it produces no duplicates, NMS is not needed.

Analogy — Dual Assignment

Think of it like practising for an exam in two ways:

- (1) Solving many practice problems of the same type (many-to-one practice) to deeply understand the concept.
- (2) On the actual exam day (inference), writing only one final clean answer per question (one-to-one).

By removing NMS, YOLOv10 gains a significant speed advantage, especially in latency-sensitive applications.

3.5 Strengths and Weaknesses of YOLOv10n

	Strengths	Weaknesses
Speed	Extremely fast; NMS-free inference	—
Size	Very small model (2–4M parameters)	Lower accuracy than larger models
Ease of use	Simple Ultralytics API; easy to train	Nano size limits representational power
Edge deploy	Excellent for embedded devices	May struggle with very small or occluded objects

4 Comparison Between RTMDet and YOLOv10n

4.1 Overview

Both models are modern, anchor-free, real-time object detectors. However, they differ in design philosophy:

- **RTMDet** prioritises *accuracy* and *flexibility*, making it suitable for research and production systems where precision matters most.
- **YOLOv10n** prioritises *speed* and *minimal resource usage*, making it ideal for deployment on edge devices or when inference latency is critical.

4.2 Comparison Table

Table 1: Side-by-side comparison of RTMDet and YOLOv10n

Criterion	RTMDet	YOLOv10n
Full Name	Real-Time Multi-scale Detector	You Only Look Once v10 (Nano)
Year	2022	2024
Source / Framework	OpenMMLab / MMDetection	Ultralytics (PyTorch)
Backbone	CSPNeXt	CSP + RepConv
Neck	Enhanced FPN	PAN-FPN
Head	Shared decoupled head	Dual head (one-to-one + one-to-many)
Anchor-Free	Yes	Yes
NMS Required	Yes (at inference)	No (removed by dual assignment)
Parameters (approx.)	~4.8M (tiny variant)	~2.3M (nano variant)
COCO mAP (approx.)	~40.9 (tiny)	~38.5 (nano)
Inference Speed (GPU)	Fast (~8–12 ms/image)	Very fast (~3–7 ms/image)
Ease of Use	Moderate (MMDetection setup)	Easy (Ultralytics one-liner)
Edge Deployment	Moderate	Excellent
Best For	Accuracy-focused tasks, segmentation pipelines	Speed-critical, resource-limited applications

Note: mAP = mean Average Precision on COCO val2017. Exact values depend on training configuration. Numbers shown are for the small/tiny/nano variants of each model.

4.3 Accuracy

RTMDet achieves slightly higher mAP than YOLOv10n on standard benchmarks (e.g., COCO). The richer CSPNeXt backbone and SimOTA label assignment give it an edge in detecting small or partially occluded objects. YOLOv10n trades some accuracy for a much smaller model size.

4.4 Speed

YOLOv10n wins on raw speed, particularly because it removes the NMS post-processing step. On a modern GPU, YOLOv10n can comfortably exceed 100 FPS, while RTMDet-tiny typically runs at 50–80 FPS depending on hardware.

4.5 Complexity and Ease of Use

- **RTMDet** requires the MMDetection framework, which has a steeper learning curve. Configuration files are written in Python-style config files that can be intimidating for beginners.
- **YOLOv10n** is available through the Ultralytics library, which offers a very simple API. Training and inference can be done in as few as *three lines of Python code*.

5 Application to Seatbelt Detection ---

5.1 Why Seatbelt Detection?

Road traffic accidents are a leading cause of death worldwide. Studies show that seatbelts reduce the risk of fatality in a crash by up to 45%. Automated seatbelt detection systems can be installed in vehicles or monitoring cameras to alert drivers or fleet managers in real time.

This is an excellent practical problem for applying object detection because:

- The objects of interest (people and seatbelts) have consistent visual patterns.
- The system must run in real time on video feeds.

- It may need to run on embedded hardware inside a vehicle (hii ya leader).

5.2 A Simple Seatbelt Detection Pipeline (implemented)

A typical seatbelt detection pipeline proceeds through three stages:

1. **Frame Extraction:** Read video frames from a camera (dashcam or in-cabin camera) at a fixed rate (e.g., 5–10 FPS for analysis).
2. **Person / Torso Detection:** Use an object detector (RTMDet or YOLOv10n) to locate the bounding box of each person (or specifically the upper torso region) in the frame.
3. **Seatbelt Classification:** For each detected person region, either:
 - Use the *same* detector if it has been fine-tuned to also detect the seatbelt strap as a separate class.
 - Use a lightweight *classifier* (e.g., a small CNN or MobileNet) on the cropped torso region to predict “seatbelt on” or “seatbelt off”.

Pipeline Diagram (Conceptual)

Video Stream → Frame Extractor → Object Detector
→ Crop Torso ROI → Seatbelt Classifier → Alert / Log

5.3 Which Model Is More Suitable?

Recommendation for Seatbelt Detection

For an **in-vehicle, real-time seatbelt detection system** running on edge hardware (e.g., Raspberry Pi 5 or Jetson Nano), **YOLOv10n is the preferred choice** due to:

- Its extremely small model size (~2.3M parameters).
- NMS-free inference, which reduces end-to-end latency.
- The simple Ultralytics API, which speeds up development.

If accuracy is the top priority (e.g., a server-side monitoring system processing recorded footage), **RTMDet** is a stronger choice.

6 Performance on Edge Hardware

6.1 What Are Edge Devices?

In computing, the term **edge** refers to devices that process data locally — close to where it is collected — rather than sending it to a remote cloud server. Common edge devices include:

- **Raspberry Pi 4 / 5:** A credit-card-sized single-board computer with a quad-core CPU. Very affordable (~\$35–80), but no dedicated GPU.
- **NVIDIA Jetson Nano / Orin Nano:** A small board with a GPU (128–1024 CUDA cores), specifically designed for AI inference workloads.
- **Google Coral Dev Board:** Uses a custom *Edge TPU* chip optimised for TensorFlow Lite models.
- **Smartphones:** Modern phones have NPUs (Neural Processing Units) that can run lightweight models quickly.

Edge deployment is challenging because these devices have:

- Limited RAM (0.5–8 GB).
- Slow processors compared to a desktop GPU.
- Strict power budgets (batteries or low-wattage supplies).

6.2 Inference Speed Comparison

Table 2: Approximate inference speed on common edge devices

Device	RTMDet-tiny (FPS)	YOLOv10n (FPS)
Desktop GPU (RTX 3060)	~70–90	~120–180
NVIDIA Jetson Orin Nano	~20–35	~40–70
NVIDIA Jetson Nano (4 GB)	~8–15	~20–35
Raspberry Pi 5 (CPU only)	~1–3	~4–8

Note: FPS values are approximate and depend on input resolution (e.g., 640×640), quantisation (FP32 vs INT8), and framework optimisations (TensorRT, ONNX). Values sourced from community benchmarks and official documentation.

6.3 Memory Usage

- **RTMDet-tiny** requires approximately **50–80 MB** of RAM for the model weights alone, plus additional memory for activations.
- **YOLOv10n** requires approximately **10–25 MB** of RAM, making it significantly more memory-efficient.

On a device like the Raspberry Pi 5 (which has 4–8 GB of RAM shared between the OS and applications), both models can fit, but YOLOv10n leaves far more memory available for other processes.

6.4 Power Efficiency

Power consumption matters in battery-powered or automotive systems. Lower power usage means longer operation and less heat generation.

- **YOLOv10n** is significantly more power-efficient because it performs fewer computations per frame (fewer floating-point operations, or FLOPs).
- **RTMDet-tiny** consumes more power but delivers higher accuracy for the energy spent.

6.4.1 Optimisation Tips for Edge Deployment

1. **Use INT8 quantisation:** Converts 32-bit floating point weights to 8-bit integers, reducing model size by $\sim 4\times$ and increasing speed by $2\text{--}4\times$ with only a small accuracy drop.
2. **Export to ONNX or TensorRT:** Both RTMDet and YOLOv10n support export to ONNX (Open Neural Network Exchange), which can then be converted to TensorRT for maximum speed on Jetson devices.
3. **Reduce input resolution:** Using 320×320 input instead of 640×640 roughly quadruples inference speed, at the cost of some accuracy on small objects.
4. **Process every N th frame:** For a 30 FPS video stream, you may only need to run detection on every 3rd frame, reducing the effective load to 10 FPS while still being responsive.

7 Conclusion

7.1 Summary of Key Insights

This report introduced two modern object detection models and compared them across several dimensions. The key takeaways are:

1. **Object detection** is a fundamental computer vision task that has broad real-world applications, from autonomous vehicles to safety monitoring.
2. **RTMDet** is a powerful, accurate detector built on the OpenMMLab ecosystem. Its CSPNeXt backbone and dynamic label assignment (SimOTA) make it an excellent choice when accuracy is the priority and sufficient computing resources are available.
3. **YOLOv10n** is an ultra-lightweight detector that removes the NMS post-processing bottleneck through its innovative dual label assignment strategy. It excels in speed, simplicity, and efficiency — making it ideal for edge deployment.

4. In the context of **seatbelt detection**, YOLOv10n is more practical for real-time, in-vehicle systems due to its speed and small footprint. RTMDet is preferable for offline or server-side analysis where higher accuracy is needed.
5. **Edge hardware** introduces constraints on memory, speed, and power. YOLOv10n handles these constraints better than RTMDet, making it the go-to choice for embedded systems like Raspberry Pi or Jetson Nano.

End of Report

References

- [1] Lyu, C., et al. (2022). *RTMDet: An Empirical High-Performance Real-Time Instance Segmentation Detector*. arXiv:2212.07784. <https://arxiv.org/abs/2212.07784>
- [2] Wang, A., et al. (2024). *YOLOv10: Real-Time End-to-End Object Detection*. arXiv:2405.14458. <https://arxiv.org/abs/2405.14458>
- [3] OpenMMLab. *MMDetection: Open MMLab Detection Toolbox and Benchmark*. <https://github.com/open-mmlab/mmdetection>
- [4] Ultralytics. *Ultralytics YOLO Documentation*. <https://docs.ultralytics.com>
- [5] Lin, T.-Y., et al. (2017). *Feature Pyramid Networks for Object Detection*. CVPR. arXiv:1612.03144.
- [6] Ge, Z., et al. (2021). *YOLOX: Exceeding YOLO Series Detectors*. arXiv:2107.08430.
- [7] Neubeck, A., & Van Gool, L. (2006). *Efficient Non-Maximum Suppression*. ICPR.
- [8] World Health Organisation. (2023). *Road traffic injuries fact sheet*. <https://www.who.int/news-room/fact-sheets/detail/road-traffic-injuries>